

Tensor Dictionary learning with sparse Tucker reconstruction

Cozzani Alberto Chiari Niccolò

June 29, 2026

The main concepts we will use are:

- Mode- n unfolding
- n -mode product
- Tucker decomposition
- HOSVD decomposition

Mode- n unfolding

The mode- n unfolding involves rearranging the entries of an N -way tensor into a matrix:

- The n -th mode is used as the first dimension of the matrix while the remaining modes are collapsed into the second dimension of the matrix.

example:

Given $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$:

- $\mathcal{Y}_{(1)} \in \mathbb{R}^{I_1 \times I_2 \dots I_N}$
- $\mathcal{Y}_{(2)} \in \mathbb{R}^{I_2 \times I_1 \dots I_N}$
- $\vdots$
- $\mathcal{Y}_{(j)} \in \mathbb{R}^{I_j \times I_1 \dots I_{j-1} I_{j+1} \dots I_N}$

n -mode product

The n -mode product of a tensor and a matrix is defined as follows:

$$\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_n \times \dots \times I_N}, \quad U \in \mathbb{R}^{J \times I_n}$$

The resulting tensor is denoted by:

$$\mathcal{B} = \mathcal{Y} \times_n U \in \mathbb{R}^{I_1 \times I_2 \times \dots \times J \times \dots \times I_N}.$$

This operation can be expressed component-wise as:

$$\mathcal{B}(i_1, \dots, i_{n-1}, \textcolor{red}{j}, i_{n+1}, \dots, i_N) = \sum_{i_n=1}^{I_n} U(\textcolor{red}{j}, \textcolor{blue}{i}_n) \mathcal{Y}(i_1, i_2, \dots, \textcolor{blue}{i}_n, \dots, i_N)$$

Properties:

Assume:

$$\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}, \mathcal{G} \in \mathbb{R}^{M_1 \times \dots \times M_N}, D_n \in \mathbb{R}^{I_n \times M_n} \quad n = 1 \dots N$$

The tensor $\mathcal{Y} = \mathcal{G} \times_1 D_1 \times_2 D_2 \times_3 D_3 \dots \times_N D_N$ is equivalent to:

- $\text{vec}(\mathcal{Y}) = (D_N \otimes \dots \otimes D_3 \otimes D_2 \otimes D_1) \text{vec}(\mathcal{G})$
- $\mathcal{Y}_{(n)} = D_n G_{(n)} (D_N \otimes \dots \otimes D_{n+1} \otimes D_{n-1} \otimes \dots \otimes D_1)^T$

Tucker decomposition

Tucker decomposition, for the case of a 3-way tensor:

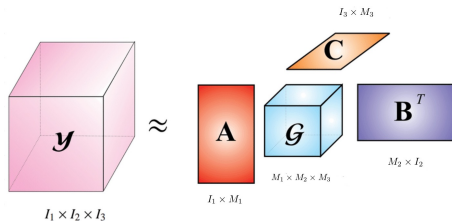


Figure: $\mathcal{Y} = \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}$

- $\mathcal{G} \in \mathbb{R}^{M_1 \times M_2 \times M_3}$: Core tensor
- $\mathbf{A} \in \mathbb{R}^{I_1 \times M_1}$, $\mathbf{B} \in \mathbb{R}^{I_2 \times M_2}$, $\mathbf{C} \in \mathbb{R}^{I_3 \times M_3}$

In general, for an N-way tensor:

$$\mathcal{Y} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \dots \times_N \mathbf{A}^{(N)}$$

Theorem(HOSVD):

Theorem(HOSVD):

A tensor $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ can be factorized as:

$$\mathcal{Y} = \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 \dots \times_N U^{(N)}$$

where $\mathcal{G} = \mathcal{Y} \times_1 (U^{(1)})^T \times_2 (U^{(2)})^T \times_3 \dots \times_N (U^{(N)})^T$ and the U 's come from the unfolding SVD,

$$\mathcal{Y}_{(k)} = U^{(k)} \Sigma^{(k)} (V^{(k)})^T, \quad k = 1 \dots N$$

Purpose of the paper

Tensor Dictionary learning, based on TUCKER decomposition, with sparsity imposed on the core tensor.

The main reasons for introducing sparsity on the core are:

- Compression of the datas;
- Enhanced clarity of the relationships between the dictionaries.

The main algorithms employed are:

- Tensor Orthogonal Matching Pursuit algorithm (TOMP);
- Gradient Descent, tensor based;

Purpose of the paper

This tensor dictionary learning algorithm is a two-stage alternating process: sparse coding and dictionary update.

- In the first stage, given the dictionaries as Tucker factors, the Tensor Orthogonal Matching Pursuit (TOMP) algorithm is used to compute the sparse core tensor;

Purpose of the paper

This tensor dictionary learning algorithm is a two-stage alternating process: sparse coding and dictionary update.

- In the first stage, given the dictionaries as Tucker factors, the Tensor Orthogonal Matching Pursuit (TOMP) algorithm is used to compute the sparse core tensor;
- In the second stage, the dictionaries are updated using a gradient descent method.

Problem formulation

The tensor dictionary learning problem is applied to the following three-mode tensor:

$$\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times I_3}, \text{ three-mode tensor.}$$

The Tucker decomposition of $\mathcal{Y}$ is:

$$\mathcal{Y} = \mathcal{X} \times_1 A \times_2 B \times_3 C \quad (1)$$

where:

- $A \in \mathbb{R}^{I_1 \times M_1}, B \in \mathbb{R}^{I_2 \times M_2}, C \in \mathbb{R}^{I_3 \times M_3}$ are the three dictionaries
- $\mathcal{X} \in \mathbb{R}^{M_1 \times M_2 \times M_3}$ is the core tensor with sparsity s .

OBS $\implies$ We suppose that the size of the core tensor $\mathcal{X}$ is larger or equal than the size of $\mathcal{Y}$ ($I_n \leq M_n, n = 1 \dots 3$)

Problem formulation

The goal is to minimize the Frobenius norm between tensor $\mathcal{Y}$ and its reconstruction:

$$\min_{\mathcal{X}, A, B, C} \left\{ \mathcal{F}(\mathcal{Y}, \mathcal{X}, A, B, C) = \|\mathcal{Y} - \mathcal{X} \times_1 A \times_2 B \times_3 C\|_F^2 \right\}$$

Problem formulation

The goal is to minimize the Frobenius norm between tensor $\mathcal{Y}$ and its reconstruction:

$$\min_{\mathcal{X}, A, B, C} \left\{ \mathcal{F}(\mathcal{Y}, \mathcal{X}, A, B, C) = \|\mathcal{Y} - \mathcal{X} \times_1 A \times_2 B \times_3 C\|_F^2 \right\}$$

with $\mathcal{M}_1$, $\mathcal{M}_2$ and $\mathcal{M}_3$ such that

$$\mathcal{X}(m_1, m_2, m_3) = 0 \quad \forall (m_1, m_2, m_3) \notin \mathcal{M}_1 \times \mathcal{M}_2 \times \mathcal{M}_3$$

where $\mathcal{M}_n = [m_n^1, \dots, m_n^{s_n}]$ denotes the subset of indices of non-zero values in the core tensor for mode- n , $n = 1, 2, 3$.

Tensor OMP

Tensor OMP is based on the equivalence of equation

$$\mathcal{Y} = \mathcal{X} \times_1 A \times_2 B \times_3 C \quad (1)$$

to the vectorized version of the tensor representation in terms of Kronecker product, i.e.

$$\mathbf{y} = (C \otimes B \otimes A)\mathbf{x}$$

where

- $\mathbf{y} \in \mathbb{R}^{l_1 \times l_2 \times l_3}$ is the vector obtained by stacking all the columns of the mode-1 $\mathcal{Y}_1$;
- $\mathbf{x}$ is a sparse vector with s non-zero entries;
- A, B, C dictionaries initialized as M_n left singular vectors of the mode- n matrices of the input tensor $\mathcal{Y}$ or randomly sample from Gaussian distribution;

Algorithm 1: Tensor-OMP

Input: $A \in \mathbb{R}^{I_1 \times M_1}$, $B \in \mathbb{R}^{I_2 \times M_2}$, and $C \in \mathbb{R}^{I_3 \times M_3}$, input signal $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, maximum number of non-zeros coefficients $t_{\max} \leq s$, max. number of iterations *maxit*.

Output: $\mathcal{X}(\mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3) = \mathcal{E}$, $\{\mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3\}$.

- ➊ Set $\mathcal{M}_i = \emptyset$, $\mathcal{R} = \mathcal{Y}$, $\mathcal{X} = 0$, $\mathbf{y} = \text{vec}(\mathcal{Y})$, *iter* = 0.
- ➋ **while** $|\mathcal{M}_1||\mathcal{M}_2||\mathcal{M}_3| \leq t_{\max}$ **or** *iter* < *maxit* **do**
 - ➌ Compute
$$[j_1, j_2, j_3] = \arg \max_{j_1, j_2, j_3} |\mathbf{R} \times_1 A(:, j_1)^T \times_2 B(:, j_2)^T \times_3 C(:, j_3)^T|.$$
 - ➍ Add j_i to $\mathcal{M}_i$ for $i = 1, 2, 3$.
 - ➎ Compute $\mathbf{e} = \arg \min_{\mathbf{z}} \|\mathbf{y} - (A(:, \mathcal{M}_1) \otimes B(:, \mathcal{M}_2) \otimes C(:, \mathcal{M}_3))\mathbf{z}\|_2^2$.
 - ➏ $\mathcal{E} = \text{tensor}(\mathbf{e})$.
 - ➐ Compute $\mathcal{R} = \mathcal{Y} - \mathcal{E} \times_1 A(:, \mathcal{M}_1) \times_2 B(:, \mathcal{M}_2) \times_3 C(:, \mathcal{M}_3)$.
 - ➑ *iter* = *iter* + 1.
- ➒ **end**
- ➓ $\mathcal{X}(\mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3) = \mathcal{E}$

Gradient Descent

The objective function:

$$\mathcal{F} = \|\mathcal{Y} - \mathcal{X} \times_1 A \times_2 B \times_3 C\|_F^2$$

is equivalent in unfolded form to:

$$\mathcal{F}_A = \|\mathcal{Y}_{(1)} - A\mathcal{X}_{(1)}(C \otimes B)^T\|_F^2,$$

$$\mathcal{F}_B = \|\mathcal{Y}_{(2)} - B\mathcal{X}_{(2)}(C \otimes A)^T\|_F^2,$$

$$\mathcal{F}_C = \|\mathcal{Y}_{(3)} - C\mathcal{X}_{(3)}(B \otimes A)^T\|_F^2.$$

For $\mathcal{X}$ **fixed**, we minimize $\mathcal{F}_A, \mathcal{F}_B, \mathcal{F}_C$ by Gradient Descent in order to update the mode- n dictionaries $A, B, C, n = 1, 2, 3$

Gradient Descent

Initialize: $A^{(0)}, \gamma > 0$ step size

- ① $k = 0$
- ② **while** $res < \epsilon_1$
 - ① **Compute** $\nabla \mathcal{F}_A$
 - ② $A^{(k)} = A^{(k-1)} - \gamma \nabla \mathcal{F}_A$
 - ③ $res = \|A^{(k)} - A^{(k-1)}\|$
 - ④ $k = k + 1$
- ③ **end**

Equivalently for dictionary B and C .

OBS $\implies$ while one dictionary is updated, the other two remain fixed.

Algorithm 2: GRADTENSOR

Initialize: dictionaries $A^{(0)}, B^{(0)}, C^{(0)}$.

Input: $\mathcal{Y}, t_{max}, maxit, \gamma, \epsilon_2$.

Repeat until convergence: $(\min_{\mathcal{X}, A, B, C} \mathcal{F} \leq \epsilon_2)$

- 1 $[\mathcal{X}, \mathcal{M}_i] = \text{TOMP}(A^{(k)}, B^{(k)}, C^{(k)}, \mathcal{Y}, t_{max}, \epsilon), \quad i = 1, 2, 3.$
- 2 Gradient descent

Return: $\mathcal{X}, A, B, C$

Experiments and results

We tested our Tensor dictionary algorithm in two different experiments:

- Synthetic data, to evaluate the algorithm's convergence.
- Medical imaging reconstruction, to assess its practical performance.
- COIL-100 Dataset

Synthetic generated tensor

In the first experiment we consider a tensor $\mathcal{Y} \in \mathbb{R}^{l_1 \times l_2 \times l_3}$ synthetically generated.

More precisely is generated from:

- $A, B, C \in \mathbb{R}^{l_n \times M_n}$ mode- n dictionaries randomly generated, $M_n = 150, l_n = 100, \quad n = 1, 2, 3$;
- $\mathcal{X}$ sparse core tensor of size $M_n = 1.5(l_n)$ whose elements follows a Gaussian distribution;

The sparsity of the core tensor has a fixed mode sparsity of $\mu = \frac{s_n}{M_n} = \frac{1}{6}$. The value of step size is fixed to $\gamma = 0.3$, the threshold parameters ϵ_1 and ϵ_2 are chosen as 10^{-6} and 10^{-4} respectively.

Experiments and results

The following graph shows the convergence behavior related to this case of study:

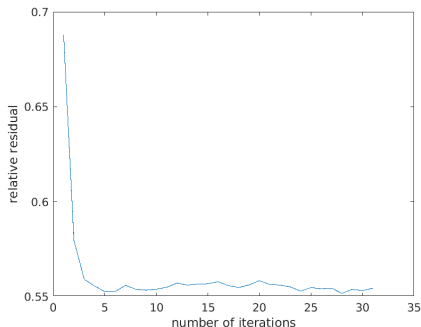


Figure: iterations vs residual error

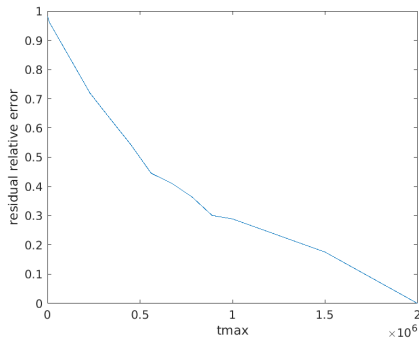


Figure: tmax vs residual error

Image reconstruction:

We apply GRADTENSOR to $\mathcal{Y} \in \mathbb{R}^{l_1 \times l_2 \times l_3}$ 3-D Human Abdomen Volume with:

- $l_1 = 151, l_2 = 125, l_3 = 141$
- $M_n = 1.5I_n$
- $\mu_1 = s_n/M_n = \frac{1}{2.2}$ and $\mu_2 = \frac{1}{3}$ different sparsity levels.

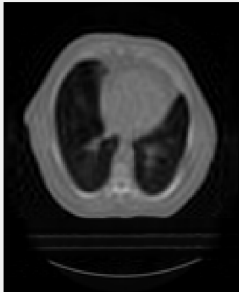
In the following slides, the reconstructed images will be shown with different levels of sparsity on the core.

Experiments and results

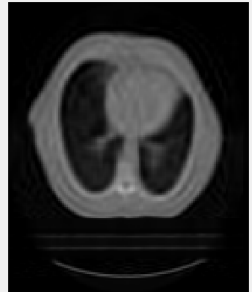
original 50th slice



reconstructed with 31% nnz



reconstructed with 12% nnz



Generalization to 4 dimensions

As dataset we took COIL-100, $\mathcal{Y}$ tensor of size $180 \times 180 \times 3 \times 60$.



Reconstructed images



Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries

Reconstructed images

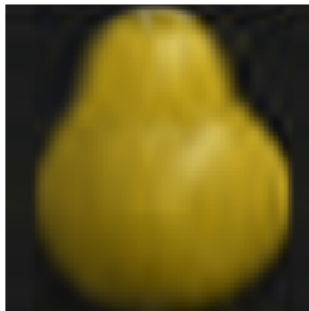


Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries



Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries

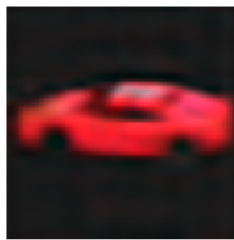
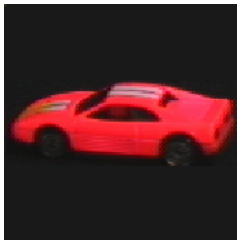


Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries

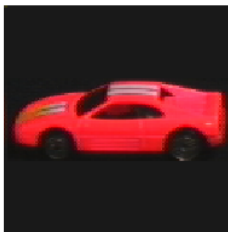


Figure: Original (left) versus reconstructed image (right), with approximately only 20 % $\mathcal{X}$ nnz entries



Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries



Figure: Original (left) versus reconstructed image (right), with approximately only 6.25 % $\mathcal{X}$ nnz entries



Figure: Original (left) versus reconstructed image (right), with approximately only 20 % $\mathcal{X}$ nnz entries

Thank you for your attention!